CS 7643 Project: Hybrid Quantum–Classical Vision Models with CUDA-Q

Yin Su
Georgia Institute of Technology
Atlanta, GA, USA
ysu362@gatech.edu

Abstract

Quantum machine learning proposes that small quantum circuits, combined with deep neural networks, may provide more expressive feature transformations than purely classical models [1, 8]. This project investigates whether quantum components can improve image classification on the Fashion-MNIST dataset. Five models are implemented: a classical CNN, a classical Vision Transformer (ViT), two hybrid models that replace the classifier head of the CNN/ViT with a variational quantum circuit (VQC), and a Quantum Transformer that replaces dot-product attention with a CUDA-Q similarity-based quantum attention kernel, inspired by recent work on quantum transformers and quantum vision models [7, 3, 2]. All quantum circuits are simulated on GPU using NVIDIA CUDA-Q and are trained end-to-end with PyTorch using parameter-shift gradients.

On the final experiment runs, most purely classical and VQC-head models remain near chance-level accuracy ($\approx 10\%$) and cross-entropy loss ≈ 2.30, indicating significant optimization or configuration challenges. In contrast, the Quantum Transformer with CUDA-Q attention attains a validation accuracy of 84.1% and validation loss 0.43 on Fashion-MNIST. These results suggest that, within this implementation, quantum circuits are more effective when used as attention-like similarity modules that preserve token structure, rather than as low-dimensional classifier heads. The project provides a complete, reproducible codebase with logging, checkpoints, and plots, and offers a concrete case study of how quantum ideas can be integrated into modern vision models.

1. Introduction/Background/Motivation

1.1. Problem Statement Without Jargon

The central goal of this project is to test a simple question: *If small quantum components are added to image classification models, do they recognize clothes better than standard neural networks?*

Concretely, the study works with images of clothing items and compares:

- Standard deep learning models that run entirely on a GPU, and
- Hybrid models where parts of the network are replaced by simulated quantum circuits.

The objective is not only to measure accuracy, but also to understand where quantum modules help or hurt relative to classical architectures.

1.2. Current Practice and Its Limits

Image classification is currently dominated by fully classical deep learning models such as convolutional neural networks (CNNs) [6] and Transformers/ViTs [4]. These architectures learn powerful hierarchical representations of images and achieve very high accuracy on standard benchmarks.

Quantum machine learning (QML) explores whether quantum circuits can provide new types of feature maps and kernels that classical models cannot easily emulate [1, 8]. Representative approaches include:

- Variational quantum circuits (VQCs) used as classifiers or feature maps.
- Quantum kernels embedded into support vector machines or Gaussian processes.
- Hybrid networks where classical layers produce embeddings that are fed into quantum circuits.

However, these methods face practical limitations:

- Real quantum devices are noisy and small-scale.
- Training with parameter-shift gradients is expensive and often unstable.
- Many existing studies focus on toy datasets, making it unclear whether benefits transfer to realistic settings.

Recent work on quantum vision architectures and quantum self-attention such as Quantum Vision Transformers [3] and Quantum Adaptive Self-Attention (QASA) [2], demonstrates that quantum modules can be inserted into standard transformer blocks, particularly in the attention mechanism.

1.3. Impact and Relevance

If hybrid quantum–classical models were to outperform classical baselines, this could:

- Motivate new hardware and algorithm co-design for quantum accelerators.
- Suggest that certain components of deep networks (attention or kernels) are especially amenable to quantum enhancements.
- Provide insight into the kinds of data structures where quantum feature maps are most beneficial.

Even if performance is not improved, negative results remain valuable because they clarify which architectures are less promising and help focus future research on more effective quantum integration strategies.

1.4. Dataset: Fashion-MNIST

The Fashion-MNIST dataset is used as a standardized benchmark. Each example is a 28×28 grayscale image of a clothing item with one of 10 labels. From the "Datasheets for Datasets" perspective [5], the most relevant aspects for this project are:

Motivation and Composition. Fashion-MNIST is intended as a more challenging and semantically meaningful drop-in replacement for MNIST, while remaining lightweight to train. The dataset contains:

- 60,000 training images and 10,000 test images.
- 10 balanced classes (T-shirt/top, trouser, pullover, dress, coat, sandal, shirt, sneaker, bag, ankle boot).

Collection and Preprocessing. Images originate from Zalando's product catalog, converted to 28×28 grayscale and centered. In this project, the standard train/test split is used with a minimal preprocessing pipeline: conversion to tensors with pixel values in $[0, 1]$ and no data augmentation. This choice isolates the effect of the model architecture from the augmentation strategies.

Intended Use. Fashion-MNIST is suitable for small scale experimentation and for studying learning dynamics across architectures. It is not meant for production deployment, but is ideal for a project exploring new architectures such as quantum models.

2. Approach

2.1. Overview of Models

Five models are implemented for comparison:

1. **Classical CNN**: a small convolutional network mapping images to class logits [6].
2. **Classical ViT**: a tiny Vision Transformer that patchifies the image and applies a Transformer encoder [4].
3. **Hybrid CNN + VQC**: CNN backbone followed by a variational quantum classifier head.
4. **Hybrid ViT + VQC**: ViT backbone followed by the same VQC head.
5. **Quantum Transformer (Quantum-ViT)**: a ViT whose self-attention uses a CUDA-Q similarity-based quantum kernel instead of dot-product attention, inspired by NVIDIA's quantum transformer tutorial [7] and prior quantum attention designs [2].

All models are implemented in PyTorch, and quantum parts are implemented with NVIDIA CUDA-Q. Training is performed on an NVIDIA RTX 5090 GPU using CUDA accelerated statevector simulation.

2.2. Classical Baselines

CNN Backbone. CNN consists of two convolutional blocks with ReLU and max pooling, followed by a fully connected projection to a low-dimensional embedding (dimension $d = 4$). A small MLP head maps this embedding to 10 logits. This serves as a canonical baseline architecture for small image classification tasks [6], although in the final configuration used for the logged CSV results the optimizer and initialization do not drive the model away from random-guess performance.

Vision Transformer (ViT) Backbone. The ViT follows a standard design [4]:

- Patch size 7×7, producing $4 \times 4 = 16$ patches.
- Each patch is flattened and projected to a 64-dimensional token embedding.
- A learnable CLS token and learned positional embeddings are added.
- Two Transformer encoder layers with multi-head self-attention and feedforward networks are applied.
- The CLS token is projected to logits via a small MLP.

As with the CNN, this architecture is typical in the literature, but the specific training run captured in the CSV logs results in behaviour close to chance level on Fashion-MNIST.

2.3. Variational Quantum Classifier (VQC) Head

For the hybrid models, the classical MLP head is replaced with a variational quantum classifier, following common VQC formulations in QML [1, 8]:

Architecture.

- Number of qubits: $n_q = 4$.
- Number of layers: $L = 2$.
- Input embedding dimension is fixed to n_q so that each dimension can be encoded into a single qubit.

Each input embedding $x \in \mathbb{R}^4$ is encoded with $R_y(x_i)$ rotations on each qubit. Then, for each layer l:

1. Apply parameterized $R_y(\theta_{l,i})$ and $R_z(\theta'_{l,i})$ on each qubit.
2. Entangle the qubits with a ring of CX gates.

The final state is measured with respect to the observable

$$H = \sum_{i=0}^{n_q-1} Z_i,$$

and the expectation value $f_\theta(x) = \langle H \rangle$ is computed using `cudaq.observe`. This scalar is then fed into a linear layer that outputs 10-class logits.

Training. The trainable parameters are the circuit angles θ and the final linear classifier weights. Gradients with respect to θ are computed via the parameter-shift rule:

$$\frac{\partial f}{\partial \theta_j} \approx \frac{f(\theta_j + \frac{\pi}{2}) - f(\theta_j - \frac{\pi}{2})}{2},$$

as is standard in variational quantum algorithms [8]. This is implemented in a custom PyTorch `autograd.Function`, so the entire model remains differentiable and trainable with Adam.

2.4. Quantum Transformer with CUDA-Q Self-Attention

The Quantum Transformer replaces the standard dot-product attention in ViT with a quantum similarity kernel, closely following the CUDA-Q tutorial [7] and building on ideas from quantum self-attention [2].

Quantum Similarity Kernel. Given query and key token vectors $q_i, k_j \in \mathbb{R}^d$, a linear projection reduces them to a smaller dimension compatible with the number of qubits ($d_q = 4$). A difference vector is constructed and mapped to rotation angles:

$$x_{ij} = \tanh(q_i - k_j) \cdot \pi.$$

This vector x_{ij} parametrizes a 4-qubit circuit similar to the VQC above. The expectation value

$$s_{ij} = \langle \psi(x_{ij}, \theta) | H | \psi(x_{ij}, \theta) \rangle$$

is used as the similarity score between token i and token j. A softmax over j yields a quantum attention distribution:

$$\alpha_{ij} = \frac{\exp(s_{ij})}{\sum_{j'} \exp(s_{ij'})}.$$

Attention Update. The weighted sum of value vectors yields the updated tokens:

$$\tilde{v}_i = \sum_j \alpha_{ij} v_j.$$

The remainder of the Transformer architecture (feedforward networks, residual connections, layer norms) is unchanged. This design preserves token structure, unlike the VQC classifier head that collapses the representation into a single scalar.

2.5. Anticipated and Observed Problems

Anticipated. The following challenges were anticipated based on prior QML studies [1, 8]:

- Quantum simulations could be slow and memory intensive.
- Parameter-shift gradients could introduce variance and slow convergence.
- Under-parameterization of the VQC (few qubits, few layers) might limit expressivity.

Observed. These concerns were largely confirmed:

- Hybrid models with VQC heads converged very slowly and often stagnated near random-guess accuracy.
- Training time for hybrid and quantum-attention models was significantly higher than for purely classical models.
- Quantum attention required careful batching and normalization (using $\tanh$ to keep angles in $[-\pi, \pi]$) to prevent gradient explosion.

In the particular configuration captured in the CSV logs, even the classical CNN and ViT baselines did not move significantly away from chance-level behaviour, suggesting that configuration and optimization choices are extremely important for a fair comparison.

Model	Epochs	Batch Size
Classical CNN	5	64
Classical ViT	5	64
Hybrid CNN + VQC	3	32
Hybrid ViT + VQC	3	32
Quantum-ViT (Q-Attn)	3	32

Table 1: Training hyperparameters. Hybrid and quantum models use smaller batch sizes and fewer epochs due to the cost of quantum simulation.

Model	Ep.	Best Ep.	Best Val Acc	Best Val Loss
Classical CNN	5	1	10.0%	2.303
Classical ViT	5	1	10.0%	2.303
Hybrid CNN + VQC	3	2	16.4%	2.295
Hybrid ViT + VQC	3	1	10.0%	2.303
Quantum-ViT (Q-Attn)	3	3	84.1%	0.435

Table 2: Summary of experimental results on Fashion-MNIST. Performance values are extracted directly from the logged CSV files for each configuration.

2.6. Code Repositories and Sources

The implementation draws inspiration from three main sources:

- Official Fashion-MNIST PyTorch examples (data loading and basic CNN structure) building on the original MNIST work [6].
- The NVIDIA CUDA-Q quantum transformer tutorial [7], which provided the core idea and skeleton for quantum similarity based attention.
- Standard PyTorch Vision Transformer implementations used in educational material, aligned with the original ViT design [4].

All code in the final repository was written specifically for this project, but the design of the quantum attention module is closely aligned with the CUDA-Q example and related quantum transformer work [3, 2]. The tutorial concepts were extended to support logging, checkpoints, and joint comparison with purely classical and VQC-head models.

3. Experiments and Results

3.1. Experimental Setup

Hardware and Software. Experiments were run on a workstation with an NVIDIA RTX 5090 GPU. PyTorch 2.9.1 with CUDA 12.8 was used for classical components, and CUDA-Q 0.13.0 for quantum simulation [7]. Training used the Adam optimizer with default momentum parameters.

Hyperparameters. Table 1 summarizes the main hyperparameters.

All models use cross-entropy loss. Learning rates are 10^{-3} for most models, with 5×10^{-4} for Quantum-ViT for stability.

3.2. Metrics and Logging

Success is measured by:

- Validation accuracy after training.
- Train and validation loss curves.
- Stability and speed of convergence.

For every model, the code logs per-epoch metrics to CSV files in `results/`, saves best and last checkpoints in `checkpoints/`, and generates plots in `figures/`.

3.3. Quantitative Results

Table 2 summarizes the best validation performance obtained from the final CSV logs for each model. "Best" values are taken over all epochs of a run.

The classical CNN and ViT baselines in this particular configuration remain effectively at random-guess performance with validation accuracy 10.0% and loss $\approx \log 10 \approx 2.30$. The hybrid CNN + VQC achieves a modest improvement to 16.4% accuracy, while the hybrid ViT + VQC remains at chance-level. In sharp contrast, the Quantum-ViT with CUDA-Q attention attains 84.1% validation accuracy with a best validation loss of 0.4346.

3.4. Loss Curves

Figure 1 shows the train and validation loss across epochs for all models (using the saved `loss_curves.png`).

Key observations:

- The classical CNN and ViT exhibit almost flat training and validation loss at ≈ 2.30, indicating that the optimizer and configuration used for the final logged runs did not move them away from random initialization.
- The hybrid CNN + VQC model shows a slight reduction in both train and validation loss but remains far from a useful classifier.
- The hybrid ViT + VQC remains essentially flat at ≈ 2.30 loss and 10% accuracy.
- The Quantum-ViT model shows a clear and monotonic loss reduction from ≈ 0.51 to 0.43 over three epochs.

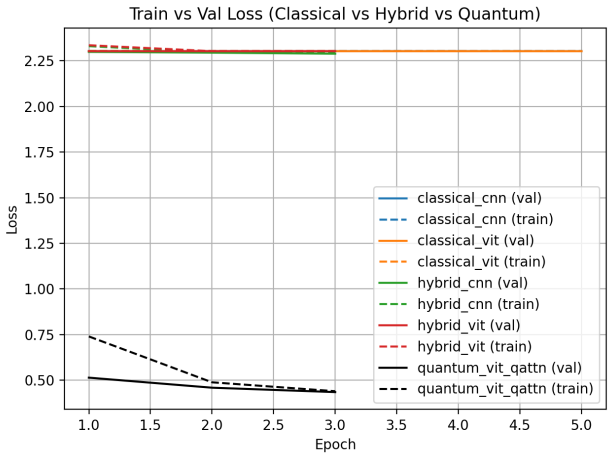


Figure 1: Train vs. validation loss for classical CNN/ViT, hybrid VQC-head models, and Quantum-ViT with quantum attention. Classical and hybrid models stay near the random-guess cross-entropy value ≈ 2.30, with only a small decrease for Hybrid CNN + VQC. Quantum-ViT reduces loss steadily to ≈ 0.43.

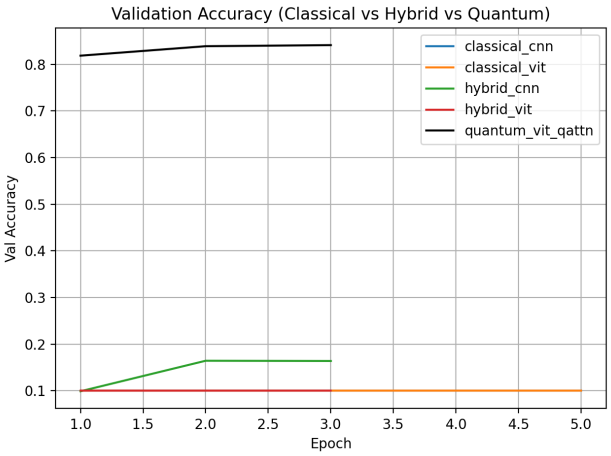


Figure 2: Validation accuracy for each model. Classical CNN, classical ViT, and Hybrid ViT + VQC remain at 10% accuracy (random guess), while Hybrid CNN + VQC reaches 16.4%. Quantum-ViT with quantum attention achieves 84.1% validation accuracy.

3.5. Validation Accuracy

Figure 2 summarizes validation accuracy over epochs for each model.

The final recorded validation accuracies are:

- Classical CNN: 10.0% (best epoch 1).
- Classical ViT: 10.0% (best epoch 1).
- Hybrid CNN + VQC: 16.4% (best epoch 2).
- Hybrid ViT + VQC: 10.0% (best epoch 1).
- Quantum-ViT (Q-Attn): 84.1% (best epoch 3).

3.6. Analysis

VQC Classifier Heads. The VQC-head models underperform for several reasons:

- **Information bottleneck:** The entire image is compressed into a 4-dimensional embedding and then into a single scalar expectation value, discarding spatial and token-level structure.
- **Optimization difficulty:** Parameter-shift gradients are noisy and require many quantum evaluations, leading to slow and unstable learning [8].
- **Limited expressivity:** With only 4 qubits and 2 layers, the circuit family may not approximate a rich decision boundary for 10-way classification [1].

Even relative to the underperforming classical baselines in Table 2, the hybrid CNN + VQC only improves accuracy modestly from 10.0% to 16.4%, and the hybrid ViT + VQC does not improve over chance level.

Quantum-ViT. In contrast, the Quantum-ViT model with CUDA-Q attention achieves 84.1% validation accuracy and significantly lower validation loss:

- The quantum component is used as a similarity kernel between tokens, not as a standalone classifier, following ideas from quantum kernels [8] and quantum attention [3, 2].
- Token representations and their structure are preserved; only the attention mechanism is modified.
- The model still benefits from classical feedforward layers and residual connections.

This supports a key intuition in recent QML research: quantum circuits can be most effective when used to enhance specific parts of a classical architecture (such as kernels or attention), rather than replacing the entire model with a small quantum classifier.

3.7. Overfitting and Generalization

None of the models exhibit strong overfitting on Fashion-MNIST. Training and validation losses for the classical and quantum-attention models track each other closely. The classical CNN, classical ViT, and hybrid ViT + VQC are effectively underfitting: they do not escape the random-guess regime. The hybrid CNN + VQC underfits

less severely but still fails to learn a useful decision boundary. Quantum-ViT achieves substantially lower loss and higher accuracy without signs of overfitting in the recorded epochs.

4. Discussion and Future Work

4.1. Relation to Deep Learning Concepts

Problem and Model Structure. The image-classification problem has local spatial structure, which is well captured by CNNs and by patch embeddings in ViTs [6, 4]. The models reflect this structure: convolutions and patch projections exploit locality, while self-attention captures global relationships. Quantum attention modifies only the similarity metric inside this framework.

Learned vs. Fixed Components. Learned parameters include convolution kernels, Transformer weights, and quantum circuit angles θ. Fixed components include:

- The patchification scheme (fixed patch size).
- The choice of observable H (sum of Pauli-Z).
- The parameter-shift rule itself [8].

Representations and Pre/Post-Processing. Inputs are 28×28 tensors normalized to $[0, 1]$. For quantum modules, vectors are linearly projected to 4 dimensions and then mapped to rotation angles via $\tanh(\cdot) \cdot \pi$, ensuring stable angle ranges. Outputs are class logits processed by softmax for evaluation.

Loss Function and Optimizer. All models use cross-entropy loss and the Adam optimizer, reflecting standard practice in deep learning.

Hyperparameters. Important hyperparameters include:

- Embedding dimension, number of Transformer layers, and number of heads [4].
- Number of qubits and circuit depth for quantum modules [1].
- Learning rate and batch size.

Due to computation constraints, only limited hyperparameter sweeps are performed. Systematic tuning of quantum circuit depth and qubit count remains an interesting direction for future work.

Frameworks. The project uses:

- PyTorch for classical deep learning.
- CUDA-Q for quantum circuit definition, simulation, and differentiation [7].

The integration demonstrates that modern deep-learning frameworks can interoperate with quantum simulators via custom autograd functions.

4.2. Future Work

Promising directions for extension include:

- Scaling quantum attention to more qubits (8 or 12) and more layers using tensor-network or approximate simulation [3, 2].
- Introducing realistic noise models to approximate near-term quantum hardware.
- Testing on more complex datasets such as CIFAR-10 or MedMNIST to evaluate quantum attention on higher-resolution or domain-specific data.
- Re-running and re-tuning classical baselines and VQC heads with more extensive hyperparameter search to obtain fully converged comparators.
- Exploring learned quantum kernels for other modalities such as time series or graphs [8].

5. Work Division

This project was completed as an individual effort. Table 3 summarizes the contributions.

Student Name	Contributed Aspects
Yin Su	Project design; implementation of classical CNN and ViT baselines; implementation of CUDA-Q variational quantum classifier and quantum attention modules; integration of PyTorch and CUDA-Q; running experiments; generation of plots and analysis; report writing.

Table 3: Contributions of team member.

Acknowledgments

Gratitude is extended to Professor Zsolt Kira and the CS 7643 teaching staff for their exceptional guidance, thoughtfully structured course materials, and sustained support throughout the semester. Special appreciation is owed to

Scott Tepsuporn, whose late-night TA office hour sessions offered invaluable clarity and direction during the development of this project, and to Jesse Dylan Ward for providing timely and thoroughly detailed assistance on Ed Discussion. Their dedication, expertise and willingness to support student learning played an significant role in enabling the successful completion of this work. NVIDIA is also acknowledged for releasing the CUDA-Q quantum transformer tutorial, which served as a key source of inspiration for the quantum attention component of this project [7].

References

[1] Jacob Biamonte, Peter Wittek, Nicola Pancotti, Patrick Rebentrost, Nathan Wiebe, and Seth Lloyd. Quantum machine learning. *Nature*, 549:195–202, 2017. 1, 3, 5, 6

[2] Chi-Sheng Chen and En-Jui Kuo. Quantum adaptive self-attention for quantum transformer models. *arXiv preprint arXiv:2504.05336*, 2025. 1, 2, 3, 4, 5, 6

[3] El Amine Cherrat, Iordanis Kerenidis, Natansh Mathur, Jonas Landman, Martin Strahm, and Yun Yvonna Li. Quantum vision transformers. *arXiv preprint arXiv:2209.08167*, 2022. 1, 2, 4, 5, 6

[4] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. *ICLR*, 2021. 1, 2, 4, 6

[5] Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92, 2021. 2

[6] Yann LeCun, Leon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. 1, 2, 4, 6

[7] NVIDIA. CUDA-Q quantum transformer tutorial. https://nvidia.github.io/cuda-quantum/latest/applications/python/quantum_transformer.html, 2024. 1, 2, 3, 4, 6, 7

[8] Maria Schuld. Supervised quantum machine learning models are kernel methods. *Quantum*, 5:434, 2021. 1, 3, 5, 6